

OCP JAVA 17 & 21 PROGRAMMER CERTIFICATION FUNDAMENTALS

OFFICIAL COURSE COMPANION

CHAPTER 24

Implementing Localization

CHAPTER OVERVIEW

This chapter covers the following exam objectives: Implement localization using locales, resource bundles, parse and format messages, dates, times, and numbers including currency and percentage values

EXAM OBJECTIVES COVERED

- ◆ Implement localization using locales, resource bundles, parse and format messages, dates, times, and numbers including currency and percentage values

Chapter 24 Implementing Localization

Exam Objectives

- Implement localization using locales, resource bundles, parse and format messages, dates, times, and numbers including currency and percentage values

24.1 Understanding localization

While developing an application it is easy to forget that the application may be used by people in different countries with different languages. At a bare minimum, an application is expected to display information meant for the consumption of the user in the language that the user can understand. An application usually produces user consumable information in the form of UI control labels such as OK and Cancel buttons, pop up messages, and, any pro-forma text that goes in the output produced upon the user's request such as PDF/Word reports. Since the regions and languages of the users of the application cannot always be known at the time of development of the application, it is almost impossible to hardcode region specific formats and texts in the application itself. Thus, instead of trying to do that, it is better for the application to not assume any language at all and use the facilities of the run time environment to localize the messages. The runtime environment can easily format information such as dates and numbers as per the local format but it cannot translate arbitrary application specific messages to the local language. To manage that, the application must use message "keys" that can be looked up into a region and language specific "resource bundle" at runtime and use the returned values to generate the messages.

An application is packaged with multiple resource bundles, but at run time, it detects

the user's region and uses the resource bundle specific to that region to translate the messages. This approach allows the application code to be written in a completely language neutral fashion and also allows the creation of the region specific resource bundles to be handled by a team that knows the local language. Such an application does not require any code change to be ported into different regions and is, therefore, called "internationalized". Refactoring an application to remove any region specific hardcoding from the application code is called **"internationalization"** or "i18n" in short (because there are 18 characters between i and n in the word internationalization) and the process of creating and packaging region specific files, i.e., resource bundles, for a particular region or language for the application is called **"localization"**.

24.1.1 Detecting a user's Locale 🙌

A user's region (the word region and country are used interchangeably in this context) and language specific information is captured by the `java.util.Locale` class. It has several methods such as `getCountry()` and `getLanguage()` that return specific details of the region that it represents. You are not expected to know the actual values that these methods return for any locale, but be aware that most of these methods return well defined codes. For example, they return `"en"` and `"US"` for language and country if the `Locale` represents a region in the US that speaks English. Note that the language is a string of 2 to 8 letters in lowercase and the country is a string of 2 or 3 letters in uppercase.

A `Locale` also captures a "category" that tells you whether this locale information is meant for customizing UI displays or for customizing textual output generated by the application. This category is represented by a static nested enum defined in `Locale` named `Category`. It has just two constants - `DISPLAY` and `FORMAT`. Category is helpful when the application needs to make a distinction between the two types of customizations. For example, an application may want to display UI controls in local language Hindi but wants to generate reports in English. For the purpose of the OCP exam, however, this distinction is not important but it should be kept in mind while building a professional application. Also remember that whenever the category is not

specified, `FORMAT` is assumed.

User's locale can be retrieved using the static methods `getDefault()` or `getDefault(Locale.Category )` of `Locale`. Many applications provide the user with an option to change their default locale. This can be done using `Locale.setDefault(Locale newLocale)` and `Locale.setDefault(Locale.Category category, Locale newLocale)`. These methods change the default locale of the JVM and any subsequent calls to `getDefault` will return the new locale.

Creating a Locale 🙌

A `Locale` object can be created using `Locale.Builder` or using overloaded static factory methods named `of` of the `Locale` class. `Locale`'s constructors are **deprecated** and can be ignored. Here are a few examples that you might see in the exam:

```
Locale enUs = new
Locale.Builder().setLanguage("en").setRegion("US").build();
Locale hi = Locale.of("hi"); //language Hindi
Locale hiIn = Locale.of("hi", "IN"); //language Hindi, country India
```

Observe that `Builder`'s has a `setRegion` (and not `setCountry`) method but this API treats region and country as the same. `Locale` also defines several `public static` constants for countries and languages for example, `Locale.US` and `Locale.CHINESE`.

An important point to note about `Locale` is that it may not always have values for all of the details of the location that it represents. For example, `Locale.CHINESE` is a `Locale` that represents just the `CHINESE` language. Calling `Locale.CHINESE.getCountry()` will return an empty string (**not** `null`). Similarly, a calling `getLanguage()` on a `Locale` built using `new Locale.Builder().setRegion("US").build()` will return `""`.

Locale overrides the `toString` method to return a string representation of the `Locale`. As per JavaDoc, it returns `language + "_" + country + "_" + (variant + "_#" | "#") + script + "_" + extensions`. Language is always lower case, country is always upper case, script is always title case, and extensions are always lower case. In the exam, you will see Locales that use only the language and the country and their string representations will be like `en_US` or `fr_CA`.

24.2 Localizing Numbers and Dates 📌

Using `NumberFormat`

People in different regions are accustomed to seeing numbers formatted differently. The JDK provides an abstract class `java.text.NumberFormatter` that can format a number as per the local standard. The following example shows how it can be used:

```
int oneMillion = 1_000_000;
Locale frFR = Locale.of("fr", "Fr");
Locale enUS = Locale.of("en", "US");
NumberFormat nfUS = NumberFormat.getCurrencyInstance(enUS);
NumberFormat nfFr = NumberFormat.getCurrencyInstance(frFR);
out.println("US:" + nfUs.format(oneMillion)); //prints US:$1,000,000.00
out.println("France:" + nfFr.format(oneMillion)); //prints France:1 000
000,00 €
nfUs = NumberFormat.getPercentInstance(enUS);
nfFr = NumberFormat.getPercentInstance(frFR);
out.println("US:" + nfUs.format(0.1)); //prints US:10%
out.println("France:" + nfFr.format(0.1)); //prints France:10 %
NumberFormat nfC = NumberFormat.getCompactNumberInstance(); //get
NumberFormat for the default locale
```

```
System.out.println("Default Locale (Compact):"+nfC.format(oneMillion));  
//prints Default Locale (Compact):10L  
nfC = NumberFormat.getCompactNumberInstance(enUS, NumberFormat.Style.SHORT);  
System.out.println("en_US (Compact):"+nfC.format(oneMillion)); //prints  
en_US (Compact):1M
```

The `NumberFormat` class has several static factory methods that can be used to create a `NumberFormat` instance. Most of them have overloaded variants that take a `Locale` as an argument. The ones that do not take a `Locale`, return an instance that formats a number according to the default locale. Note that the instances returned by these methods are **not thread safe**.

The exam does not expect you to know the exact format in which numbers are printed for different locales but it does expect you to know the currency symbols for US Dollar(\$), Euro(€), and GB Pound(£).

Parsing Numbers

A `NumberFormat` instance can also be used to parse a String containing a number written in locale specific manner. For example, `nfC.parse("1M")`, where `nfC` points to the same `NumberFormat` instance used in the above example, will return a `Long` containing the value `1000000`. Be aware that the return type of the `parse` method is `Number` but it may actually return a `Double` or a `Long` depending on whether the string has digits after decimal or not. Also remember that this method can potentially throw a `java.text.ParseException`, which is a checked exception.

Using `DateTimeFormatter`

To format the `LocalDate/Time` objects of the `java.time` package, you need to use the `java.time.format.DateTimeFormatter` class. It has three different methods,

`ofLocalizedDate`, `ofLocalizedTime`, and `ofLocalizedDateTime`, which can be used for formatting `LocalDate`, `LocalTime`, and `LocalDateTime` respectively. They require a `FormatStyle` argument. `DateTimeFormatter` also has a `ofPattern` method, which can format the `LocalDate/Time` objects for any given `Locale`. This method requires a `FormatStyle` and a `Locale` as arguments. For example:

```
DateTimeFormatter dfLocal =  
    DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT);  
out.println(dfLocal.format(LocalDate.of(2024, 12, 1))); //prints 01/12/24  
DateTimeFormatter dfFr = DateTimeFormatter.ofPattern("dd MMM uuuu",  
    frFR);  
out.println(dfFr.format(LocalDate.now())); //prints 06 mai 2024
```

You are not expected to know the formats of various `FormatStyle` options for the exam.

JDK has a `java.text.DateFormat` class, which is similar to the `java.text.NumberFormat` and can be used for format dates according the given `Locale`. However, since this class was created long before the new `Date/Time` API, it can only format `java.util.Date` objects. Unfortunately, it hasn't been enhanced to format `java.time.LocalDate/Time` objects.

Parsing Dates

`DateTimeFormatter` has several methods that can parse a string into a `LocalDate/Time` objects. Technically, parsing dates is mentioned in the objectives but parsing dates is quite complicated and we haven't seen anyone getting a question on this. If you have time, you may go through the JavaDoc of the `DateTimeFormatter` class to learn more about this topic.

24.3 Localizing text

24.3.1 Internationalizing an app using ResourceBundle 🙌

While developing an internationalized application, all user facing text or words used by the application are identified and listed in file along with their translations in the language that the development team knows in the form of name-value pairs. For example, let's say an application asks the user for their name and then prints that name with a greeting. Something like this:

```
public class HelloApp{
    public static void main(String args){
        Console c = System.console();
        String name = c.readLine("What is your name? ");
        String greetings = "Hi";
        System.out.println(greetings+" "+name);
    }
}
```

The application contains two user facing strings - "What is your name?" and "Hi". To internationalize this application, these two strings must not be hardcoded. Instead, the code must use keys to look up the the actual phrases at run time. This is done as follows:

```
import java.util.ResourceBundle;
public class HelloApp{
    public static void main(String args){
        ResourceBundle rb = ResourceBundle.getBundle("helloapp");
        Console c = System.console();
        String name = c.readLine(rb.getString("What is your name?")+ " ");
        String greetings = rb.getString("Hi");
        System.out.println(greetings+" "+name);
    }
}
```


There are several moving parts in the above code. I will explain all of them but let's get it to run first. If you compile and run it, you will get an exception with the following message:

```
java.util.MissingResourceException: Can't find bundle for base name  
helloapp, locale en_IN
```

You may see a different locale name in the output because it depends on the locale of your machine. Now, create a file named `helloapp.properties` in the same folder that contains `HelloApp.class` with the following contents:

```
What\ is\ your\ name?=What is your name?  
Hi=Hi
```

If you run it now, it will work as expected. The output isn't any different from the previous version but we have accomplished an important thing in the above code. We have managed to look up the keys used in the program and show the resulting values to the user. Although, it still prints messages in English, the application has now been internationalized! All we need to do now is to localize it for the locales in which we want to run this application. But before we do that, let me explain a few points about the above code:

1. A `ResourceBundle` object contains locale-specific objects. Every resource bundle has a name and the program uses that name to load the relevant files that contain locale specific entries. Once the entries in the files are loaded in the resource bundle, the program can just lookup up a key in that resource bundle and use the returned locale specific value. A resource bundle is not limited to containing just strings but for the purpose of the exam, we will stick to strings only.
2. `java.util.ResourceBundle` is an abstract class but the JDK also provides a couple of concrete subclasses. One of those subclasses is `PropertyResourceBundle`, which can load key-value pairs from a properties file. However, instead of directly instantiating the `PropertyResourceBundle` class, we use `ResourceBundle`'s `getBundle` method because this method is smart enough to figure out exactly which properties files are needed to create the desired resource bundle based on the user's locale, which is what we want.

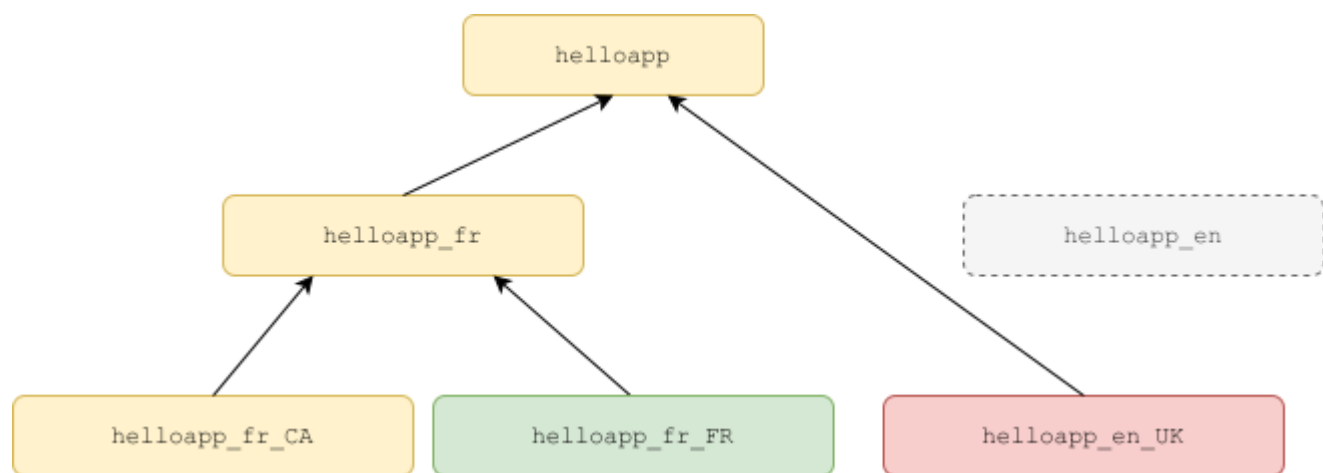
2. The logic to determine the properties files needed to create a resource bundle is important and I will explain it shortly, but in this example, since we don't have any locale specific properties file, the `getBundle` method just loads a properties file with the same name as that of the resource bundle (but with extension `.properties`), i.e., `helloapp.properties`.
3. The name of the resource bundle and the corresponding properties file follows the same convention as the one followed by classes. Therefore, while locating a bundle named `helloapp`, the `getBundle` method expects `helloapp.properties` to be located directly on the classpath. If the name of the bundle were `test.helloapp`, then `helloapp.properties` would have to be present in the `test` folder and the `test` folder would have to be on the classpath.
4. The name of the keys in the properties file can be anything but the code must use the same key to loop up the value. Therefore, it is a common practice to use strings containing text that makes sense in the language that the dev team understands as keys. Using keys such as `key1`, `key2`, makes it hard to tie a key to the text that is meant to be shown to the user for that key.
5. The backslashes (`\`) in the properties files are required to escape the space character.

24.3.2 Understanding Resource Bundles

Resource bundles are hierarchical, meaning, a resource bundle always has a parent bundle unless the bundle is at the top of the hierarchy, in which case, it is called the "base bundle". The hierarchy of resource bundles is established using a set of resource bundles whose names start with a common base name, such as `helloapp` used in the previous example. The remaining part of the name of a resource bundle is created using various attributes of a locale such as the language and the country separated by underscores. These attributes must always be in the following order:

```
baseName + "_" + language + "_" + script + "_" + country + "_" + variant
```

For the purpose of the exam, only the language and the country are important but script and variant behave similarly. So, for example, there can be a hierarchy of resource bundles with names `helloapp`, `helloapp_fr`, and `helloapp_fr_CA`, where the bundle named `helloapp` is at the top (grandparent) and the bundle named `helloapp_fr_CA` is at the bottom. Note that a bundle named `helloapp_fr_FR` will also be a part of this hierarchy but it will be on a new branch splitting from `helloapp_fr`. Similarly, the bundle named `helloapp_en_UK` will also be a separate branch splitting off of `helloapp_en` but if `helloapp_en` does not exist then `helloapp_en_UK` will be the immediate descendant of `helloapp`. This is illustrated in the following image:



All of these resource bundles belong to the same family of resource bundles because they share the same base name, i.e., `helloapp`.

Understanding the hierarchy of resource bundles is important because when a key is not found in a particular bundle, it is automatically searched for in the parent bundle up to the base bundle. When a key is not found in any of the ancestor bundles either, a `MissingResourceException` is thrown. So, if `helloapp_fr` is queried for a key that it does not have, its parent bundle `helloapp` will be checked and if `helloapp` too doesn't have that key then a `MissingResourceException` will be thrown. Note that the search for a key only propagates up the hierarchy and never to a child or a sibling.

So now, based on the above discussion, can you tell what resource bundle we should

have if, for example, we want to localize our app for French speaking people in Canada? Right, `helloapp_fr_CA` indeed. But remember that the values from this bundle will not be used for French speaking people in France because their locale would be `fr_FR` and they would search for the keys in `helloapp_fr_FR`. If you check the above graph, `helloapp_fr_CA` is a sibling and not a parent of `helloapp_fr_FR`. However, if we have resource bundle named `helloapp_fr` **instead of** `helloapp_fr_CA`, then the values from this bundle can be used for all French speaking locales because `helloapp_fr` is a parent of `helloapp_fr_CA` as well as `helloapp_fr_FR`.

24.3.3 Creating resource bundles using properties files 🙌

The `ResourceBundle` class has several overloaded **static** `getBundle` methods out of which two are particularly important - `getBundle(String baseName)` and `getBundle(String baseName, Locale locale)`. The former returns a `ResourceBundle` for the user's default locale while the later returns a `ResourceBundle` for the given locale. The `baseName` is the common name of the all the resource bundles of a family as explained before.

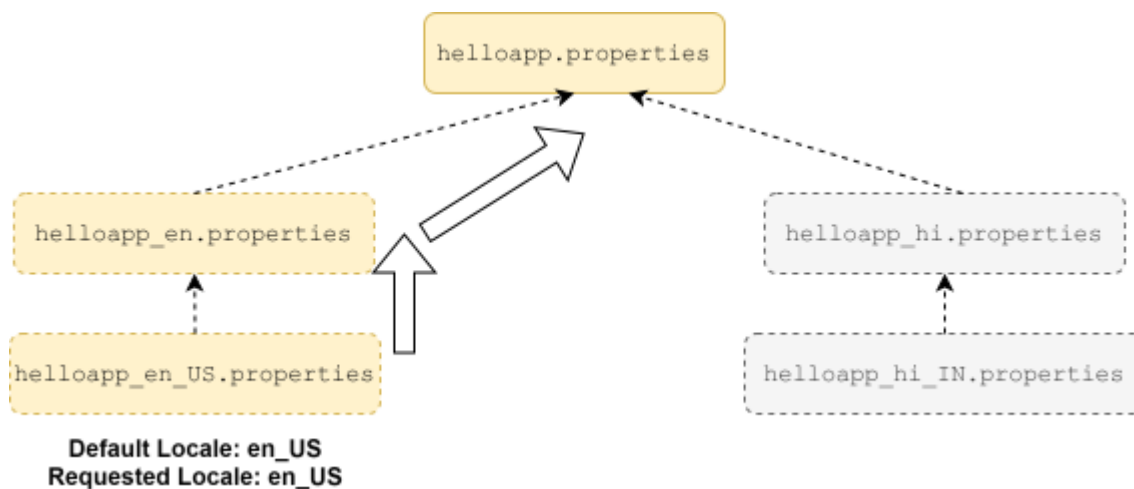
Requesting a resource bundle for the default locale 🙌

The `getBundle` methods automatically read a properties file whose name matches with the name of the requested resource bundle (provided the properties file is on the classpath) and create a `PropertyResourceBundle` instance that contains the entries defined in the properties file. So, for example, assuming that the user's default locale is `en_US`, the call to `ResourceBundle.getBundle("helloapp")`, will look for a properties file named `helloapp_en_US.properties` file in the classpath and will load the entries into the `helloapp_en_US` bundle. Remember that since resource bundles are hierarchical, creation of a `ResourceBundle` triggers the creation of its parent bundle as well. Therefore, once `helloapp_en_US` bundle is created, its parent bundle `helloapp_en` would also be created automatically in a

similar fashion. Thus, it will look for `helloapp_en.properties` to create the `helloapp_en` bundle. Similarly, since the parent of `helloapp_en` is `helloapp`, `helloapp.properties` will also be loaded.

If, while creating the hierarchy of the resource bundles, at any stage, the `getBundle` method is not able to find the matching properties file, it tries to create the parent bundle of the requested bundle and returns that parent bundle instead. So, if `helloapp_hi_IN.properties` is not available, `getBundle("helloapp", Locale.of("hi", "IN"))` method will try to create the parent bundle, i.e., `helloapp_hi` as per the process described above and return this bundle instead.

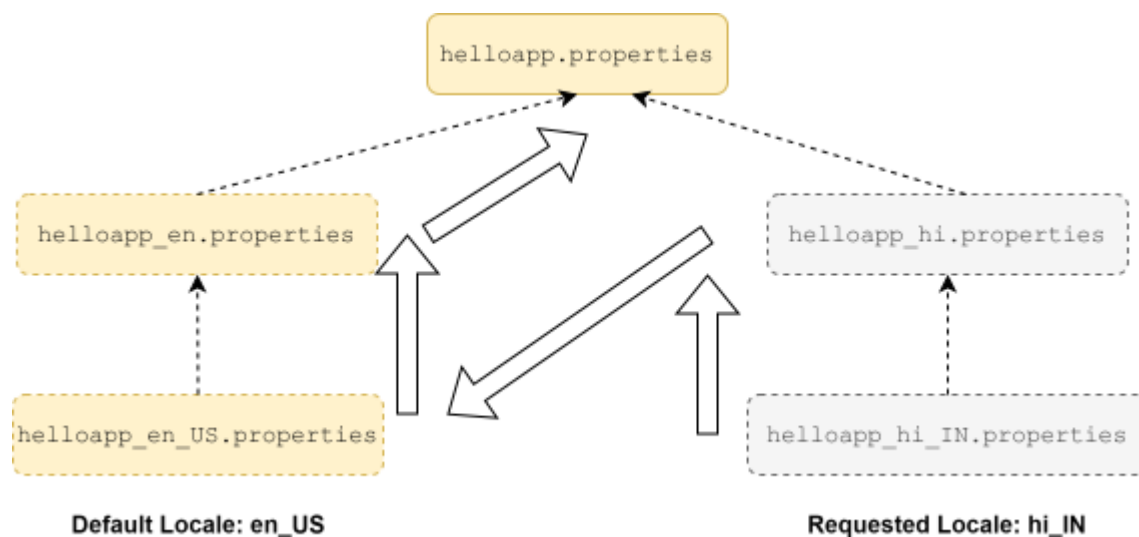
Effectively, the call to `getBundle` causes the creation of the hierarchy from the requested bundle up to the base bundle (but not the children or siblings) using properties files. The following image illustrates the search path that `getBundle` takes in this example:



Requesting a resource bundle for the non-default locale 📌

The process for creating a requested bundle for a non-default locale is almost the same as above. The only difference is that when `getBundle` is not able to find any locale specific properties file, it doesn't immediately return the base resource bundle. Before returning the base bundle, it tries to find and return the bundle for the default

locale. So, for example, let's say the default locale of the user is en_US and you request a bundle for hi_IN locale by calling `getBundle("helloapp", Locale.of("hi", "IN"))`. Now, if this method is not able to find any properties file that matches hi_IN locale (i.e. `helloapp_hi_IN.properties` or `helloapp_hi.properties`), it will try to create and return the bundle for the default locale, i.e., `helloapp_en_US`. If it is not able to find any properties file for en_US locale either, it will return the base bundle, i.e., `helloapp`. The following image illustrates the search path that `getBundle` takes in this example:



At this point, you might be curious to know what happens if `getBundle` is not able to find any properties file that it can use to create any bundle at all. Well, in that case it throws a `MissingResourceException`.

The exam does not try to trick you on the location of the properties files. Unless the problem statement states otherwise, you can safely assume that the properties file given in the question is on the classpath and can be loaded by a `ResourceBundle`. You just need to check the name of the properties file and the name of the resource bundle being requested using the `getBundle` method.

24.3.4 Looking up a key in a ResourceBundle 🙌

If you have read the previous two sections carefully, this section will be like a walk in the park. To look up a key in a `ResourceBundle`, you need to first get hold of a `ResourceBundle` that you are interested in using one of the `getBundle` methods and then call one of the following three `getXXX` methods on that resource bundle:

`String getString(String key)` : Gets a string for the given key from this resource bundle or one of its parents.

`String[] getStringArray(String key)` : Gets a string array for the given key from this resource bundle or one of its parents.

`Object getObject(String key)` : Gets an object for the given key from this resource bundle or one of its parents.

Only the first one is important for the exam and you have already seen the example that uses this method earlier. As you must have observed in the short descriptions of the above methods, they return a value from this bundle or from one of its parents.

The exam tests you on whether you are able to figure out the value for a key for a non default locale when the key is not present in the properties file for the non-default locale but is present in the properties file of the default locale. You need to remember that loading a resource bundle and looking up the key in a resource bundle are two separate and independent steps. If a properties file for the non-default locale is present, then a resource bundle will be created using that file as explained in the previous section. The key lookup in the non-default bundle will happen in this non-default bundle hierarchy only and if the key is not found in it, a `MissingResourceException` will be thrown. The key will **NOT** be looked up in the resource bundle hierarchy of the default locale. The resource bundle hierarchy of the default locale is used only when no locale specific resource bundle is found.

Ideally, the application code shouldn't need to pass in a `Locale` to get the bundle. It should just pass the base name of the

bundle and get the bundle for the default locale of the application because passing a locale defeats the purpose of `i18n`.

The application should split the entries into multiple bundles though for easier management of the entries. For example, you could create separate resource bundles for each of the functionally different parts of an application. Note that an application may have any number of resource bundles.

24.3.5 Formatting Messages using `MessageFormat` 🙋

Showing messages that are grammatically correct in different locales is tricky. For example, a sentence such as "Your balance is 100 points." would be grammatically correct in many languages such as English. However, it would be grammatically incorrect in many other languages such as Hindi, in which the correct form would be, "Your balance 100 points is". Thus, if the program creates this message using string concatenation such as `String msg = rb.getString("yourbalanceis")+" "+amount+" "+rb.getString("points");`, the resulting string may look odd in some locales.

The `java.text.MessageFormat` class can be used to solve this problem. Instead of hardcoding string concatenation, you can specify parameters in the message string and, at run time, format the string by passing values for those parameters. For example:

```
ResourceBundle rb = ResourceBundle.getBundle("helloapp");  
String message = MessageFormat.format(rb.getString("yourbalanceis"), 100);  
System.out.println(message);
```


The above code uses the static `String format(String pattern, Object... arguments)` method of `DateFormat`. The value for `yourbalanceis` key will be `Your balance is {0}.` in `helloapp_en_us.properties` but `Your balance {0} is.` (but in Hindi, of course) in `helloapp_hi_IN.properties`.

Using the `DateFormat` class is quite simple. The parameters are specified using curly braces and besides the parameter index (indexing starts with zero), you may specify how you want to format it as well. For example, a raw message could be, `"On {0, date, short}, your balance was {1, number, currency}"` and it can be formatted using an array of two objects as arguments like this: `DateFormat.format(msg, new Object[]{new Date(), 100});` This will return `"On 10/05/24, your balance was $100.00."`. A serious issue with `DateFormat` is that it was created before the introduction of the `java.time` package and hasn't been updated to work with the new `LocalDate/Time` classes. So, if you want to use dates as arguments, you have to use `java.util.Date` objects.

24.3.6 Quiz 🙋

Q. Assume that the default locale of the application is `en_US` and the following two properties files are present on the classpath:

```
//in file msgs_es_MX.properties
Hi=Hola
thankyou=gracias
```

and

```
//in file msgs_en_US.properties
Hi=Hi
Regards=Regards
```

What will be the output of the following code snippet?

```
ResourceBundle rb1 = ResourceBundle.getBundle("msgs");
System.out.println(rb1.getString("Hi")); //1

ResourceBundle rb2 = ResourceBundle.getBundle("msgs", Locale.of("es",
"MX"));
System.out.println(rb2.getString("Hi")); //2

ResourceBundle rb3 = ResourceBundle.getBundle("msgs", Locale.of("MX"));
System.out.println(rb3.getString("Hi")); //3

ResourceBundle rb4 = ResourceBundle.getBundle("msgs", Locale.of("hi",
"IN"));
System.out.println(rb4.getString("Hi")); //4

ResourceBundle rb5 = ResourceBundle.getBundle("msgs", Locale.of("US"));
System.out.println(rb5.getString("Regards")); //5

ResourceBundle rb6 = ResourceBundle.getBundle("msgs", Locale.of("MX"));
System.out.println(rb6.getString("Regards")); //6

ResourceBundle rb7 = ResourceBundle.getBundle("msgs", Locale.of("es",
"MX"));
System.out.println(rb7.getString("Regards")); //7
```

Answer: Hi Hola Hi Hi Regards Regards <MissingResourceException> (ignoring new lines)

Observe the following points:

1. Since the default locale is `en_US`, whenever you request a bundle without specifying the locale (as in `//1`) or for a locale for which no locale specific properties file is present (as in `//3`, `//4`, `//5`, and `//6`), the bundle for the default locale, which is created using `msgs_en_US.properties` will be returned. Thus, `//1`, `//3`, and `//4` will print `Hi`, and `//5` and `//6` will print `Regards`.
2. Asking for a bundle for `es_MX` as in `//2` and `//7` will return the `es_MX` bundle because `msgs_es_MX.properties` is present. The default bundle will not be used at all if you look for keys in this bundle. Therefore, `//2` will print `Hola` but `//7` will throw a `MissingResourceException` because the `es_MX` bundle doesn't contain the key `regards`.

In the exam, you may encounter a badly written question that does not explicitly specify the default locale of the application. In such a case, you may assume the default locale to be `en_US` but **only if** it is absolutely necessary to arrive at the correct answer.

24.4 Exercise

1. Write a program that detects and prints the details such as the language and country of its default locale. Change the default locale to something else.
2. Input a number with decimals and print its localized string version for the default locale as if it were a monetary amount and also as if it were a percentage value.
3. Create a family of resource bundles for two different locales (one for the default and one for any another locale) with base name `com.abc.reports` using properties files. Add a key named `goodbye` to all of them. Retrieve the locale specific value for this key from the program for the default as well as the other

3. locale.